USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Two Mysteries, One Confirmed Bug, and a Novel That Called It First

UM-NATOS-029

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-029	1.1 · 2026-08-18	**Superseded — both mysteries resolved in [UM-NATOS-030](UM-NATOS-030-one-bit.md)**	Internal

> **Both mysteries in this report have one cause, and it is not in here.** `DMA_OUTLINK_START` was defined as bit 30, which is `OUTLINK_RESTART`; START is bit 29. Every DMA transfer resumed the old descriptor chain instead of starting the new one, so the panel received a displaced copy of a correct framebuffer.
>
> The FIFO path never touches that register, which is why the view "healed" when a spurious timeout disabled DMA, why gfxrogue and `hog draw` (180 px, DMA-sized fills) appeared to repair it by provoking that timeout sooner, and why the `draw` application (20 px, FIFO-sized) never did. §10's conclusions about continuous interleaved drawing are wrong; the mechanism is in UM-NATOS-030 §3.
>
> Kept as written, because the eleven eliminated theories in §4 are still eliminated and the reasoning that missed the cause is the point.

1. Abstract

This report covers a session that set out to explain why the 3D view is garbled when it opens, and instead found something else entirely: **the display's DMA engine has been dead since a few seconds after every boot, on every build, for an unknown number of weeks.** Every pixel this kernel has drawn in that time went out through a 64-byte FIFO.

That is a genuine defect, it is fixed, and §6 covers it. It is not, however, an explanation for the thing we were looking for. Both original mysteries survive the session intact:

- **Mystery One** — the 3D view opens garbled, heals itself after ten to sixty seconds, and can be healed on demand by running `gfxrogue`, a program whose entire purpose is to *fail* to draw anything.
- **Mystery Two** — if the board is simply left alone for five to seven minutes after boot, the view opens correctly the first time, with no program involved.

Eleven theories were tested and eliminated. §4 keeps the list, because a negative result that nobody records gets tested again in three weeks.

There is also a joke the author did not intend to make, and §2 is about that.

2. The novel that described the bug before we found it

Alongside the twenty-eight engineering reports, the author has been writing a novel — *The NatOS Organization: A Novel of System Architecture* — in which the kernel's subsystems are staff in an office. It exists at `docs/used_medias/um_1.txt` and in forty chapters beside it.

In its first chapter, written before any of this session's debugging, the Scheduler is refusing the Touch Controller a context switch:

> *"I need a context switch," Touch Controller pleaded over the departmental channel. "My PENIRQ line just asserted!"*
>
> *"Impossible," Scheduler replied, not looking up from his lists. "The Display Driver has priority inheritance. The 3D raycast renderer is in the middle of a DMA transfer. Interrupting now would tear the framebuffer."*

Every clause of that excuse turned out to be a live issue in this session. **Two of them turned out to be false**, and the source says so in both cases.

The novel says	The instruments say
the renderer holds the display	`dlock hold ms=7965` against ~7,860 ms of uptime — it holds the draw lock essentially without interruption
Touch loses the argument	`drawskip` climbing into the thousands; `vp_fill()` uses `display_try_lock()` and simply gives up
nobody is contending	true, and misleading. `cont=0` — nothing ever blocks on the lock, because everything else yields rather than waits
"*has priority inheritance*"	**false.** nat-os has **ageing**. `mutex.c` contains no mention of priority at all
"*in the middle of a DMA transfer*"	**false.** DMA has been disabled since seconds after boot. It was all going out over the FIFO

The Display Driver's excuse was load-bearing, and it was wrong about both mechanisms. It was defending a transfer that was not happening, using a scheduling feature this kernel does not implement.

2.1 Ageing is not priority inheritance

`task.c:284` draws the distinction itself, which is how the claim was settled:

> *The base priority is never modified: ageing is a property of the SELECTION, not of the task, so a task that finally runs returns to its declared priority automatically rather than needing to be restored. That distinction is what keeps this separate from priority inheritance, which really does change a task's priority and really does have to undo it.*

They solve overlapping problems and are not interchangeable. **Ageing** lifts a task that has waited too long, regardless of why it waited or who holds what — `g_age_rescues` counts 4,513 such decisions in a single dump, so the policy is doing constant real work. **Inheritance** lifts the specific task that is *blocking* you, and must undo the change afterwards.

The irony is that the mechanism this kernel actually has is the better fit for the scene. Inheritance would be close to useless here, because it only helps when a high-priority task **blocks** on a lock — and in nat-os almost nothing blocks. `cont=0` is that fact. Applications call `display_try_lock()` and walk away; the only task that

ever blocks on the panel is the display task itself (`dblk disp/apps/touch/shell=3630/0/0/0`), and ageing is what rescues it.

A related check, since it was raised by the same passage and is the kind of thing that looks like a defect: `vp_fill()` takes `display_try_lock()` and then calls `display_fill_rect()` , which takes `draw_lock()` on the same mutex. That is **not** a recursive-acquire bug. `mutex_t` carries `owner` and `depth` , and `mutex_try_lock()` increments `depth` when the caller already owns it (`mutex.c:32`). `draw_lock()` further guards its telemetry on `depth == 1u` , so nested acquisitions cannot corrupt the hold-time measurement — which is what makes the 7,965 ms figure above worth quoting.

This is offered as a curiosity rather than a method. But it is a real one: an author writing fiction about their own scheduler produced a more accurate description of the contention than the status line did, because the status line reported `cont=0` and everyone read that as "no contention problem" rather than "nobody is even allowed to contend."

---

3. The two mysteries, stated precisely

3.1 Mystery One — the view garbles, and a failing program fixes it

Opening the 3D view shortly after boot produces a scrambled image. It heals after roughly ten to sixty seconds. It can also be healed immediately by launching **`gfxrogue`** .

`gfxrogue` (`tools/app_gfx_rogue.vasm` , 76 bytes, nineteen VM instructions) exists to prove an application cannot draw outside its viewport. It asks for the whole 240×320 panel in red, then white, then asks for a rectangle at origin (60000, 60000). In pseudocode:

```
forever:
    paint the ENTIRE screen red      -> clipped to a 180x14 strip
    paint the ENTIRE screen white    -> clipped to the same strip
    paint at (60000, 60000)          -> refused outright, nothing drawn
```

All three requests are lies, and the kernel refuses all three as stated. What the user sees is a small band near the bottom flickering red and white. That flickering strip is the containment test passing.

The mystery is that this program — which mostly does not draw, and when the 3D view is open usually loses the race for the draw lock and does not draw *at all* — reliably repairs the display.

3.2 Mystery Two — uptime alone fixes it

Observed this session, and the more useful of the two: **leave the board running for five to seven minutes and the 3D view opens correctly the first time.** No program, no intervention.

That makes the variable *uptime*, which is a far smaller thing to search than "drawing". The one attempt to measure it produced an invalid early capture (§7.4) and has not yet been repeated.

A related observation from the same session, not yet reproduced under instrumentation: at low uptime the view may not open **at all**, rather than opening wrongly. Those are different faults and the distinction matters.

---

4. Eleven theories, all eliminated

Each was tested against hardware, not argued away.

#	Theory	How it died
1	DMA fallback to FIFO mid-blit	`dma=320/0` at the time; also §7.1
2	Touch task scheduling	identical across all four polling configurations
3	Application overdraw	killed every app; no change
4	Panel signal integrity	40/20/10 MHz changes appearance only
5	Chrome column overwriting the view	`APP_VIEW_Y0` 224 == `RAY_VIEW_H` 224; static assert added
6	Movement catch-up burst	fixed via `raycast_open()` ; symptom unchanged
7	Touch calibration	fixed; symptom unchanged
8	Arena/framebuffer overlap	`fb 0x3ffbcd90..0x3ffd7190` , arenas from `0x3ffd75b0`
9	Panel window desync	`resync` — one CASET/PASET/RAMWR — no change
10	Window-setup *frequency*	`resyncn 500` — a burst of them — no change
11	CPU starvation	`hog` — spins like `task_apps` , draws nothing — no change

Two further probes bracketed the drawing itself:

- **`fifopoke`** — a single 16-byte transfer on the FIFO path. No change.
- **`stripn 200`** — `gfxrogue` 's exact clipped rectangle, same colours, same window setups, same DMA bursts, issued directly from the shell. **No change.**

`stripn` is the important negative. It reproduced everything `gfxrogue` puts on the wire and repaired nothing, which retires the entire transport-side family of theories in one stroke.

4.1 The one thing that did work

The tests above cover a 2×2 with a hole in it:

	burst	continuous
draws	`stripn` ✗	never tested
doesn't draw	—	`hog` ✗

`gfxrogue` is the missing cell: drawing, forever, interleaved with the renderer. `hog draw` — which spins *and* fills continuously, using `display_try_lock()` so it behaves like an application — **repaired the view.**

Credit where due: the author identified this gap, not the instruments. The phrasing was "*gfxrogue is spamming gfx code, the kernel has to let the tasks take turns*", and it was correct where four measured theories in a row had not been.

That result is real but it is not yet an explanation. It says the repair needs continuous interleaved drawing. It does not say why the picture is wrong without it.

---

5. Render or transport?

With `hog draw` giving a reliable on/off switch for the first time, the obvious question became answerable: is the *framebuffer* wrong, or only the picture?

It took three attempts to ask it properly.

Attempt one compared `fbsum` in both states and found large differences — `equal-to-first` 10,304 versus 718 — which was reported as proof of a render fault. It was nothing of the sort. **The camera walks continuously**, so two samples fifteen seconds apart differ by construction. A second run had the numbers flip direction (215 versus 4,256), which is what a confound looks like when it stops flattering the hypothesis.

Attempt two added `camfreeze`: hold the viewpoint still while the renderer keeps drawing every frame. Distinct from `dfreeze`, which stops the display task and freezes the buffer along with it. With the camera pinned at `cell 3,11 frac 829,362`, frames advanced 992 → 1061 in six seconds (~11.5 fps), and the framebuffer summary was identical in both states.

Attempt three replaced the summary with a real checksum. `fbsum` now emits `fbhash`, FNV-1a over all 53,760 pixels, because the zero and equal-to-first counts are a *sample* and two different pictures can share them. Result: `fbhash=0x5b985ce0`, unchanged for 76 seconds and several hundred rendered frames.

And the run proved nothing, because the picture never garbled during it. A constant hash from a frozen camera rendering a correct scene is just a renderer working. The instrument was finally right and the bug declined to appear.

Two things came out of it anyway:

- The camera was `in open space` in every sample. The flat-colour signature at `raycast.c:501` — "*buried itself in a wall, and every column rendered as one flat colour*" — is a documented failure of this renderer, and it is **not** what is happening here. `campos` asks `wall_at()` directly rather than inferring it from how flat the picture looks.
- 76 seconds with the camera frozen and it never garbled once. Suggestive that the fault needs the viewpoint to be moving — but the view had already been open ~16 seconds before freezing, inside the window where it heals anyway, so this is a lead and not a result.

---

6. What was actually found: DMA has been dead this whole time

`dmastat`, on a running system:

```
transfers 110507  timeouts 1  <- DMA is OFF, everything is on the FIFO path
```

`spi2_dma_tx()` bounded its wait on **wall clock** at 2,000,000 cycles (~25 ms at 80 MHz). That is generous for a 480-byte transfer and far too short for the *wait*, because the wait can span a scheduling round trip. A display task descheduled mid-transfer times out on a DMA engine that is working perfectly.

A timeout is not a dropped frame. It sets `g_dma_ok = 0` **permanently**, and every transfer for the rest of the run falls back to the 64-byte FIFO. The `transfers` counter freezing at 110,507 across captures seven minutes apart is that: nothing has used DMA since it died.

This is why a full-view blit costs **55.8 ms** against 21.7 ms to compose and 2.3 ms to march. Getting the picture to the glass is roughly 70% of every frame, on a path that was supposed to be the slow fallback.

Bound raised to 40,000,000 (~500 ms). A genuinely stuck engine is still caught; it costs half a second once instead of 25 ms.

6.1 A second defect, not yet fixed

When `spi2_dma_tx()` returns 0, `spi_tx()` re-sends **the whole buffer** over the FIFO. The engine may already have shifted some or all of those bytes out. They reach the panel twice and offset everything after them in the RAMWR stream — a torn frame, by construction, at exactly the moment of a timeout.

Left alone deliberately so the timeout fix could be tested as a single variable. It should be fixed regardless: either skip the re-send, or re-issue the window first.

---

7. Instruments that lied — the tally continues

UM-NATOS-028 §8 kept this list. It has grown.

7.1 The boot self-test's `dma=N/0`

`display_init()` prints the DMA counters as part of the boot report, and it reads zero timeouts every time. It runs **before the scheduler starts**, so it measures the one condition in which nothing can preempt anything — precisely the condition under which the bug cannot occur. This number was read as proof the timeout never fires, and it retired theory #1 for most of the session.

7.2 Circular reasoning about the fix

Worse than the above, and self-inflicted. The bound was raised to 40,000,000, `dmastat` was checked, it read **timeouts=0**, and that was taken as evidence the raise had been unnecessary. The raise is *why* it read zero. The change was then reverted on that reasoning, and the defect stayed in the tree for several more hours until the original bound was back in place long enough to read `1`.

A guard can only be shown unnecessary by measuring it in its absence.

7.3 A summary mistaken for a checksum

`fbsum` reported a zero count, an equal-to-first count, and the first pixel of four rows. Two states matching on all six numbers was called "identical framebuffers". They are a sample; different pictures can share them. Now it emits `fbhash` over every pixel.

7.4 A test harness that skipped its own wait

The uptime comparison script accepted an `EARLY` delay parameter and never used it. The "early" capture therefore ran at ~12 seconds uptime, while the kernel was still booting — `frames 0 columns 0`, and `act/tap/open=1/0/0` showing the desktop still in launcher mode because the `view3d` command went out before the shell was reading. The entire early column of that comparison was void.

7.5 An empty column that looked like data

The same harness printed a `frames` field parsed by a matcher that returned the empty remainder after the bare token `frames`. The column showed `7840` — which was `equal-to-first` shifted over — and appeared to be a plausible, unchanging frame count for thirty-eight consecutive samples.

7.6 `raycast_framebuffer()` returns a boolean

Carried over from the previous session and still worth its place: a diagnostic used it as a pointer and stored through `0x00000001`. `raycast_fb_ptr()` now exists for the pointer, and the naming is the whole defence.

---

8. Instruments added this session

All read-only or opt-in; none change any existing code path unless invoked.

Command	Purpose
`dmastat`	the DMA counters from a **running** system, not from boot
`campos`	camera cell, sub-cell position, heading, and whether `wall_at()` says it is inside geometry
`camfreeze`	hold the viewpoint still while the renderer keeps drawing
`view3d`	open/close the 3D view from the terminal, so the moment of opening belongs to the test
`hog` / `hog draw`	spin like `task_apps`; `draw` also fills continuously
`resyncn`	a burst of window setups, no pixels
`stripn`	`gfxrogue`-shaped fills without `gfxrogue`
`fbsum`	now emits `fbhash`, FNV-1a over all 53,760 pixels

`view3d` matters more than it looks. Every previous attempt to catch the startup fault failed the same way: the view was opened by hand, seconds passed before anything could be armed, and the picture had already healed. A startup failure needs the instrument to exist before the first frame does.

---

9. What is established, and what is not

Established:

- DMA times out spuriously within seconds of every boot and is disabled for the run. Fixed.
- The renderer holds the draw lock essentially continuously; applications use `display_try_lock()` and skip rather than wait, so `cont=0` means "nobody is permitted to contend", not "no contention exists".
- Continuous interleaved drawing (`hog draw`, `gfxrogue`) repairs the view. A burst of the same drawing does not. Neither does spinning without drawing.
- The camera is **not** inside geometry when the view is wrong. That documented failure mode is not this one.

- Nothing sent to the panel — window setups, FIFO transfers, identical clipped rectangles — repairs the view on its own.

Not established:

- Whether the fault is in the renderer or in the transport. Three attempts, no answer; the one properly controlled run never reproduced the bug.
- What changes between 30 seconds and 7 minutes of uptime.
- Whether the low-uptime symptom is "opens garbled" or "does not open", which may be two faults being reported as one.
- Why continuous drawing repairs it. `hog draw` is a reliable switch, not an explanation.

---

10. Next

1. **Re-run the uptime comparison** with the harness fixed, capturing all forty status counters at 45 s and at 7 minutes, anchored to an observation of the panel at both points.
2. **Fix the double-send** in the DMA timeout fallback (§6.1).
3. **Catch it with `view3d` + `camfreeze` armed before the first frame.** If it garbles with a frozen camera, `fbhash` across the heal answers §5 outright. If it stays clean, the fault requires camera movement — which is itself the finding.
4. **Ask whether the DMA fix changed anything.** The whole investigation ran on a FIFO-only display path. That is now different, and every measurement above was taken under the old condition.

The Scheduler was right about the priorities and wrong about the transfer. The Touch Controller never got its context switch. `drawskip` is at 4,844 and climbing.